

Documentation Maintenance — VizHome

Procédures de maintenance, surveillance, sauvegardes et résolution d'incidents pour la plateforme VizHome en production.

Accès aux systèmes

Système	Accès	URL
API Django	SSH + Docker	ssh root@<IP_SERVEUR>
Django Admin	Web (auth admin)	https://api.vizhome.fr/admin/
Traefik Dashboard	Web (Basic Auth)	https://traefik.vizhome.fr
MinIO Console	Web	https://minio.vizhome.fr
Logs (Sentry)	Web	https://sentry.io

Surveillance quotidienne

Vérifier la santé des services

```
cd /opt/vizhome/backend-vizhome

# Statut global de tous les containers
docker compose -f docker-compose.prod.yml --env-file .env.prod ps

# Healthcheck API
curl -fsS https://api.vizhome.fr/health/ready
# Réponse attendue : {"status":"ok","checks":{"postgres":"ok","redis":"ok"}}

# Healthcheck liveness (ne teste pas les dépendances)
curl -fsS https://api.vizhome.fr/health/live
```

Indicateurs à surveiller

Indicateur	Normal	Alerte
Statut containers	Up X (healthy)	unhealthy OU Exited
Réponse /health/ready	HTTP 200, "status":"ok"	HTTP non-200
Workers Celery	celery worker: Up	Exited
Espace disque	< 80%	≥ 85%
RAM consommée	< 70%	≥ 85%

Vérifier l'espace disque

```
df -h
du -sh /opt/vizhome/backend-vizhome/backups/
docker system df
```

Logs

Consulter les logs en temps réel

```
cd /opt/vizhome/backend-vizhome

# API Django
docker compose -f docker-compose.prod.yml --env-file .env.prod logs -f api

# Worker Celery
docker compose -f docker-compose.prod.yml --env-file .env.prod logs -f celery

# Celery Beat (tâches planifiées)
docker compose -f docker-compose.prod.yml --env-file .env.prod logs -f celery-beat

# Traefik
docker compose -f docker-compose.prod.yml --env-file .env.prod logs -f traefik

# Tous les services (limité aux 100 dernières lignes)
docker compose -f docker-compose.prod.yml --env-file .env.prod logs --tail=100
```

Niveaux de log importants

Niveau	Action requise
INFO	Aucune — informatif
WARNING	Surveillance — dégradation possible
ERROR	Investigation requise
CRITICAL	Intervention immédiate

Sentry (monitoring production)

Si `SENTRY_DSN` est configuré, toutes les exceptions remontent automatiquement vers Sentry avec :

- Stack trace complète
- Contexte utilisateur (depuis le JWT)
- Tags Celery (nom de tâche, tentatives)

Configurer des **alertes email** dans Sentry pour les erreurs `ERROR` et `CRITICAL` .

Sauvegardes

Sauvegarde automatique PostgreSQL (quotidienne)

Le script `scripts/backup_postgres.sh` est lancé via cron.

Configuration cron :

```
crontab -e
# Ajouter :
0 3 * * * cd /opt/vizhome/backend-vizhome && ./scripts/backup_postgres.sh >> /var/log/vizhome-backup.log
2>&1
```

Le script :

1. Exporte la base de données compressée (`pg_dump | gzip`)
2. Nomme le fichier `backup-YYYYMMDD-HHMMSS.sql.gz`
3. Stocke dans `./backups/`
4. Supprime les backups de plus de **30 jours**

Sauvegarde automatique MinIO (hebdomadaire)

```
crontab -e
# Ajouter :
0 4 * * 0 cd /opt/vizhome/backend-vizhome && ./scripts/backup_minio.sh >> /var/log/vizhome-backup-
minio.log 2>&1
```

Le script effectue un **mirror complet** du bucket MinIO. Rétention : 7 jours.

Sauvegarde manuelle PostgreSQL

```
cd /opt/vizhome/backend-vizhome

# Dump compressé
docker compose -f docker-compose.prod.yml --env-file .env.prod exec postgres \
    pg_dump -U vizhome vizhome | gzip > backup-$(date +%Y%m%d-%H%M%S).sql.gz
```

Restaurer PostgreSQL depuis un backup

```
# 1. Arrêter l'application pour éviter les écritures
docker compose -f docker-compose.prod.yml --env-file .env.prod stop api celery celery-beat

# 2. Restaurer
gunzip -c backup-YYYYMMDD-HHMMSS.sql.gz | \
    docker compose -f docker-compose.prod.yml --env-file .env.prod exec -T postgres \
    psql -U vizhome -d vizhome

# 3. Redémarrer
docker compose -f docker-compose.prod.yml --env-file .env.prod start api celery celery-beat
```

Sauvegarde hors-site (recommandé)

Copier les backups vers un stockage distant :

```
# Exemple avec rclone vers Backblaze B2
rclone sync /opt/vizhome/backend-vizhome/backups/ b2:vizhome-backups/

# Cron hebdomadaire hors-site
0 5 * * 0 rclone sync /opt/vizhome/backend-vizhome/backups/ b2:vizhome-backups/ >> /var/log/rclone.log
2>&1
```

Mise à jour de l'application

Procédure de mise à jour standard

```
cd /opt/vizhome/backend-vizhome

# 1. Faire une sauvegarde avant mise à jour
./scripts/backup_postgres.sh

# 2. Récupérer les nouveaux commits
git pull origin main

# 3. Rebuilder et redémarrer
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build

# 4. Vérifier le bon redémarrage
docker compose -f docker-compose.prod.yml --env-file .env.prod ps
curl -fsS https://api.vizhome.fr/health/ready
```

L'entrypoint applique **automatiquement les nouvelles migrations** au démarrage.

Mise à jour des dépendances Python

```
# Editer src/requirements.txt
nano src/requirements.txt

# Rebuilder l'image
docker compose -f docker-compose.prod.yml --env-file .env.prod build api celery
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --force-recreate api celery
```

Gestion des tâches Celery

Vérifier l'état des workers

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec celery \
    celery -A config inspect active
```

Voir les tâches en file d'attente

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec redis \
    redis-cli LLEN celery
```

Purger la file d'attente Celery (urgence uniquement)

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec celery \
    celery -A config purge
```

Relancer un render bloqué (manuellement)

```
# Via le shell Django
docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
    python manage.py shell -c "
from apps.renderers.tasks import generate_render
```

```

from apps.renders.models import Render

render = Render.objects.get(id='<RENDER_UUID>')
render.status = 'pending'
render.save()
generate_render.delay(str(render.id))
print('Render relancé')
"

```

Tâches planifiées (Celery Beat)

Gérer les tâches depuis Django Admin : https://api.vizhome.fr/admin/django_celery_beat/periodictask/

Tâche	Fréquence	Vérification
Reset compteurs rendus	1er du mois à 00h00	UserStats.renders_this_month remis à 0
Snapshot métriques admin	Tous les jours à 00h05	AdminDailySnapshot créé
Nettoyage uploads forum	Tous les jours à 03h00	Images orphelines supprimées

Gestion de la base de données

Accéder au shell PostgreSQL

```

docker compose -f docker-compose.prod.yml --env-file .env.prod exec postgres \
    psql -U vizhome -d vizhome

```

Requêtes de supervision utiles

```

-- Taille de la base
SELECT pg_size_pretty(pg_database_size('vizhome'));

-- Tables les plus volumineuses
SELECT tablename, pg_size_pretty(pg_total_relation_size(tablename::regclass)) AS size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(tablename::regclass) DESC
LIMIT 10;

-- Connexions actives
SELECT count(*) FROM pg_stat_activity WHERE state = 'active';

-- Requêtes lentes (> 5s)
SELECT pid, now() - query_start AS duration, query
FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '5 seconds';

```

Appliquer des migrations manuellement

```

# Voir les migrations en attente
docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
    python manage.py showmigrations --list | grep '\[ \]'

```

```
# Appliquer
docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
python manage.py migrate
```

Gestion du stockage MinIO

Accéder à la console MinIO

```
https://minio.vizhome.fr
Identifiants : MINIO_S3_ACCESS_KEY / MINIO_S3_SECRET_KEY (définis dans .env.prod)
```

Vérifier l'espace occupé

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec minio \
mc du local/vizhome-media
```

Nettoyer les images forum orphelines manuellement

```
docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
python manage.py cleanup_forum_orphan_uploads --dry-run # preview

docker compose -f docker-compose.prod.yml --env-file .env.prod exec api \
python manage.py cleanup_forum_orphan_uploads # suppression réelle
```

Incidents courants et résolutions

Container API en état `unhealthy`

Symptômes : `docker compose ps` affiche `(unhealthy)` , API non accessible.

```
# 1. Consulter les logs
docker compose -f docker-compose.prod.yml --env-file .env.prod logs --tail=50 api

# 2. Redémarrer le container
docker compose -f docker-compose.prod.yml --env-file .env.prod restart api

# 3. Si persistant : forcer une recréation
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --force-recreate api
```

Rendus IA bloqués en statut "En cours"

Cause possible : crash du worker Celery pendant une génération.

```
# 1. Vérifier le statut du worker
docker compose -f docker-compose.prod.yml --env-file .env.prod ps celery

# 2. Redémarrer le worker
docker compose -f docker-compose.prod.yml --env-file .env.prod restart celery
```

```
# 3. Passer les renders bloqués en "failed" via Django Admin
# Admin → Renders → Filtrer par status=processing → Mettre à jour en batch
```

Base de données inaccessible

```
# Vérifier l'état de Postgres
docker compose -f docker-compose.prod.yml --env-file .env.prod ps postgres
docker compose -f docker-compose.prod.yml --env-file .env.prod logs postgres

# Vérifier l'espace disque (Postgres peut refuser de démarrer si plein)
df -h

# Redémarrer Postgres
docker compose -f docker-compose.prod.yml --env-file .env.prod restart postgres
```

Certificat TLS expiré

Traefik renouvelle automatiquement les certificats Let's Encrypt. Si un problème survient :

```
# Vérifier les logs Traefik
docker compose -f docker-compose.prod.yml --env-file .env.prod logs traefik | grep -i acme

# Forcer le renouvellement (supprimer le fichier ACME et redémarrer)
docker compose -f docker-compose.prod.yml --env-file .env.prod exec traefik \
    rm -f /letsencrypt/acme.json
docker compose -f docker-compose.prod.yml --env-file .env.prod restart traefik
```

Espace disque critique

```
# Voir les plus gros dossiers
du -sh /opt/vizhome/backend-vizhome/backups/
docker system df
docker images

# Nettoyer les images Docker inutilisées
docker image prune -f

# Nettoyer les anciens backups (garder les 14 derniers)
ls -t /opt/vizhome/backend-vizhome/backups/*.sql.gz | tail -n +15 | xargs rm -f
```

Tokens JWT expirés en masse

Si l'horloge système dérive, tous les tokens JWT peuvent devenir invalides.

```
# Vérifier l'heure du serveur
date

# Synchroniser NTP
systemctl restart systemd-timesyncd

# ou
ntpdate -u pool.ntp.org
```

Opérations de maintenance planifiée

Avant une maintenance

1. Avertir les utilisateurs via la bannière admin (ou email)
2. Prendre une sauvegarde complète
3. Tester la procédure sur un environnement de staging

Maintenance en mode dégradé

Pour placer l'application en maintenance (page d'erreur 503) :

```
# Via Traefik : activer une règle de maintenance dans traefik/dynamic/maintenance.yml
# (à créer selon les besoins)
```

Mise à niveau majeure de base de données

Pour les migrations complexes (ex: rename de colonnes sur tables volumineuses) :

1. Prendre une sauvegarde complète
2. Tester la migration sur un dump local
3. Planifier une fenêtre de maintenance
4. Appliquer en production avec suivi des logs

Métriques et monitoring avancé (optionnel)

Prometheus + Grafana

Pour activer les métriques Prometheus, ajouter dans les dépendances :

```
django-prometheus==2.x
```

Traefik expose déjà ses métriques Prometheus sur le port `8082` (interne).

Uptime Kuma

Outil léger pour le monitoring externe (vérification depuis l'extérieur) :

```
# Configurer des monitors pour :
# - https://api.vizhome.fr/health/ready
# - https://vizhome.fr
# - https://cdn.vizhome.fr
```

Contacts d'urgence

Rôle	Email	Usage
Support technique	dev@vizhome.fr	Incidents généraux
Sécurité	security@vizhome.fr	Faillles de sécurité
Hébergeur	(selon fournisseur)	Problèmes d'infrastructure